

CS 233 OBJECT ORIENTED PROGRAMMING AND DESIGN PATTERNS (OOPDP)



Shamama Anwar

Assistant Professor

Department of Computer Science and Engineering

Birla Institute of Technology, Mesra.

shamama@bitmesra.ac.in

9431103880

Module - IV

Exception Handling and GUI Design

Exception Handling

- When to use exception handling
- Java exception hierarchy
- finally block
- Stack unwinding
- Chained exceptions, Declaring new exception types, Assertions, try with resources.

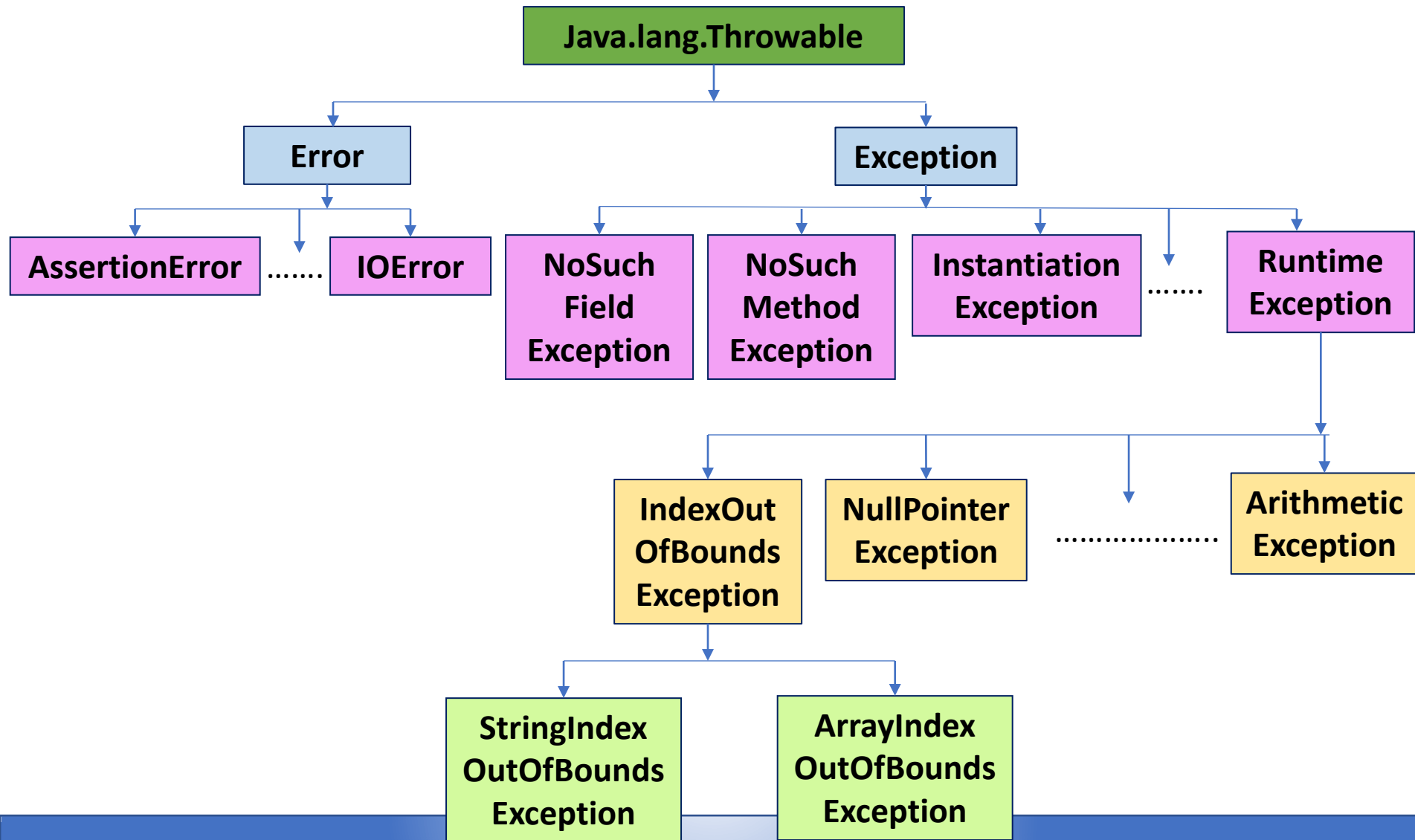
GUI Design

- Simple I/O with GUI
- Basic GUI Components
- GUI Event handling, Adapter classes, Layout managers, using Panels

Exception Handling

- An *exception* is an error condition that changes the normal flow of control in a program.
- When a program encounters an unexpected termination or fault, it is called an exception. Eg: when we try and divide by 0 we terminate abnormally.
- Exception handling gives us another opportunity to recover from the abnormality.
- Sometimes we might encounter situations from which we cannot recover like Outofmemory. These are considered as errors.

Exception Hierarchy



Exception Types

Checked Exception

- Exceptions which are checked for during compile time are called checked exceptions.
- A checked exception either must be caught by a method, or must be listed in the throws clause of any method that may throw or propagate it.
- The compiler will issue an error if a checked exception is not caught or asserted in a throws clause
- Example: `NoSuchFieldException`, `NoSuchMethodException`

Exception Types

Unchecked Exception

- Exceptions which are not checked for during compile time are called unchecked exception.
- The exceptions raised, if not handled will be handled by the Java Virtual Machine. The Virtual machine will print the stack trace of the exception indicating the stack of exception and the line where it was caused.
- The only unchecked exceptions in Java are objects of type `RuntimeException` or any of its descendants
- Example: `ArithmeticException`, `ArrayIndexOutOfBoundsException`

Some Exception Examples

ArithmeticException

```
int a = 50/0;
```

NullPointerException

```
String s = null;  
System.out.println(s.length());
```

NumberFormatException

```
String s = "abc";  
int a = Integer.parseInt(s);
```

ArrayIndexOutOfBoundsException

```
int a = new int[5];  
a[6] = 12;
```

Exception Handling Techniques

- try catch
- throw
- throws
- finally

try catch

- Can be placed within any method that can throw an exception.
- The statements to be tried for exceptions are put in a try block.
- Immediately following the try block is the catch block.
- Catch block is used to catch any exception raised from the try block.
- When an exception occurs, processing continues at the first catch clause that matches the exception type.

Exception Handling Techniques

```
try {  
    ... normal program code  
}  
catch(Exception e) {  
    ... exception handling code  
}
```

Exception Handling - try catch

```
import java.util.Scanner;
class test
{ public static void main(String args[])
  {
    int a, b, c;
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter two numbers:");
    a = sc.nextInt();
    b = sc.nextInt();
    c = a / b;
    System.out.println("Result:"+c);
  }
}
```

Exception in thread main java.lang.ArithmeticException:/ by zero

Exception Handling - try catch – Example 1

```
import java.util.Scanner;
class test
{ public static void main(String args[])
  {
    int a, b, c;
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter two numbers:");
    a = sc.nextInt();
    b = sc.nextInt();
    try
    {
      c = a / b;
      System.out.println("Result:"+c);
    }
    catch(ArithmeticException e)
    { System.out.println("ERROR"+e); }
    System.out.println("After try catch");
  }
}
```

Exception Handling - try catch – Example 2

```
class test
{ int a, b;
  test()
  {
    a=5;
    b=0;
  }
  test(int a, int b)
  {
    this.a = a;
    this.b = b;
  }
  int div()
  {
    return (a/b);
  }
}
```

Exception Handling - try catch – Example 2

```
class demo
{ public static void main(String args[])
  { int a, b;
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter two numbers:");
    a = sc.nextInt();
    b = sc.nextInt();
    test t1 = new test(a,b);
    try
    {
      System.out.println("Result:"+t1.div());
    }
    catch(ArithmeticException e)
    { System.out.println("ERROR"+e); }
    System.out.println("After try catch");
  }
}
```

Exception Handling – Multiple catch

- Multiple catch clauses can be written against a try block.
- As soon as an exception occurs, the first appropriate catch clause is responsible for handling the exception.
- At a time only one Exception occurs and at a time only one catch block is executed.
- All catch blocks must be ordered from most specific to most general i.e. catch for `ArithmeticException` must come before catch for `Exception` .

Exception Handling – Multiple catch

```
class Test
{ public static void main(String args[])
{
    try
    {
        int a[]=new int[5];
        a[5]=30/0;
    }
    catch(Exception e)
    {System.out.println("common task completed");}
    catch(ArithmeticException e)
    {System.out.println("task1 is completed");}
    catch(ArrayIndexOutOfBoundsException e)
    {System.out.println("task 2 completed");}
    System.out.println("rest of the code...");
}
}
```

Exception Handling – Multiple catch

```
public class Test
{ public static void main(String args[])
  { try
    {   int a[]=new int[5];
        a[5]=30/0;
    }
    catch(ArithmeticException e)
        {System.out.println("task1 is completed");}
    catch(ArrayIndexOutOfBoundsException e)
        {System.out.println("task 2 completed");}
    catch(Exception e)
        {System.out.println("common task completed");}
    System.out.println("rest of the code...");
  }
}
```


Program

- WAP to create an array and ask user to input values into it. Then ask the user to choose an array index number and divide the entire array by the value at that array location. Use exception handling mechanism wherever required.

Exception Handling Techniques

throw

- It is used to explicitly throw an exception.
- An object of exception needs to be created before they are thrown.
- throw is more useful when we want to throw a user defined exception.

```
throw new ArithmeticException();
```

- Since it is a transfer statement , we can not place statements after throw statement. It leads to compile time error **Unreachable code**

Exception Handling – throw – Example 1

```
public class TestThrow1
{
    static void validate(int age)
    {
        if(age<18)
            throw new ArithmeticException("not valid");
        else
            System.out.println("welcome to vote");
    }

    public static void main(String args[])
    {
        validate(13);
        System.out.println("rest of the code...");
    }
}
```

Exception Handling – throw – Example 2

```
public class TestThrow1
{
    static void validate(int age)
    {
        try {
            if(age<18)
                throw new ArithmeticException();
            else
                System.out.println("welcome to vote");
        }
        catch(ArithmeticException ae)
        {
            System.out.println("Invalid age !!");
        }
    }

    public static void main(String args[])
    {
        validate(13);
        System.out.println("rest of the code...");
    }
}
```

Exception Handling – throw – Example 3

```
public class TestThrow1
{ public static void main(String args[])
    { method1(); }
  static void method1()
    { System.out.println("In method1, calling method2");
      method2();
      System.out.println("Returned from method2"); }
  static void method2()
    { System.out.println("In method2, calling method3");
      try { method3(); }
      catch(Exception e) {System.out.println("Exception handled"+e);}
      System.out.println("Returned from method3"); }
  static void method3()
    { System.out.println("In method3");
      throw new rithmeticException("Testing throw");
      System.out.println("Returning from method3"); }
}
```

Exception Handling – nested try

```
class Nest { public static void main(String args[])
{ try { try { System.out.println("Inside block1");
           int b =45/0; System.out.println(b); }
    catch(ArithmeticException e1){
        System.out.println("Exception: e1"); }
    try { System.out.println("Inside block2");
        int b =45/0; System.out.println(b); }
    catch(ArrayIndexOutOfBoundsException e2){
        System.out.println("Exception: e2"); }
    System.out.println("Just other statement"); }
    catch(ArithmeticException e3){
        System.out.println("Arithmetic Exception");
        System.out.println("Inside parent try catch block"); }
    catch(ArrayIndexOutOfBoundsException e4){
        System.out.println("ArrayIndexOutOfBoundsException");
        System.out.println("Inside parent try catch block"); }
    catch(Exception e5){ System.out.println("Exception");
        System.out.println("Inside parent try catch block"); }
    System.out.println("Next statement.."); } }
```

Exception Handling Techniques

throws

- It is added to the method signature to let the caller know about what exceptions the called method can throw.
- It is the responsibility of the caller to either handle the exception or it can also pass the exception (by specifying throws clause)

```
public void f1() throws new ArithmeticException()  
{ . . . }
```

Exception Handling – throws – Example 1

```
class M
{
    void method()throws IOException
    {    throw new IOException("device error"); }
}
public class Testthrows2
{
    public static void main(String args[])
    {
        try
        {    M m=new M();
            m.method();
        }
        catch(Exception e){System....("exception handled");}
        System.out.println("normal flow...");
    }
}
```


Exception Handling – throws – Example 2

```
class M
{
    void method() throws IOException
    {
        System.out.println("device operation performed");
    }
}

class Testthrows3
{ public static void main(String args[]) throws IOException
    {
        M m=new M();
        m.method();
        System.out.println("normal flow...");
    }
}
```

Exception Handling – throws – Example 3

```
class Testthrows1{
    void m()throws IOException{
        throw new IOException("device error");    }
    void n()throws IOException{
        m();    }
    void p(){
        try{
            n();}
        catch (Exception e) {System....("exception handled");}
    }
    public static void main(String args[]){
        Testthrows1 obj=new Testthrows1();
        obj.p();
        System.out.println("normal flow...");
    }
}
```

Exception Handling – throws – Example 4

```
public class TestThrow1
{ public static void main(String args[])
    { method1(); }
  static void method1()
    { System.out.println("In method1, calling method2");
      method2();
      System.out.println("Returned from method2"); }
  static void method2()
    { System.out.println("In method2, calling method3");
      try { method3(); }
      catch(Exception e){System.out.println("Exception handled"+e);}
      System.out.println("Returned from method3"); }
  static void method3() throws Exception
    { System.out.println("In method3");
      if(b==0)
        throw new ArithmeticException("Testing throw");
      else
        System.out.println("Result:"+a/b);
    } }
```

Exception Handling Techniques

finally

- The finally block is always executed irrespective of whether an exception is thrown from within the try block or not.

```
try { . . . } catch (Exception e) { . . . } finally { . . . }
```

Exception Handling – finally – Example 1

```
class TestFinallyBlock
{
    public static void main(String args[])
    {
        try
        {   int data=25/5;
            System.out.println(data);
        }
        catch (NullPointerException e)
        {   System.out.println(e); }
        finally
        {   System.out.println("Finally block"); }
        System.out.println("rest of the code...");
    }
}
```

Exception Handling – finally – Example 2

```
class TestFinallyBlock
{
    public static void main(String args[])
    {
        try
        {   int data=25/0;
            System.out.println(data);
        }
        catch (NullPointerException e)
        {   System.out.println(e); }
        finally
        {   System.out.println("Finally block"); }
        System.out.println("rest of the code...");
    }
}
```

Exception Handling – finally – Example 3

```
class TestFinallyBlock
{
    public static void main(String args[])
    {
        try
        {   int data=25/0;
            System.out.println(data);
        }
        catch (ArithmeticException e)
        {   System.out.println(e); }
        finally
        {   System.out.println("Finally block"); }
        System.out.println("rest of the code...");
    }
}
```

Exception Handling – finally – Example 4

```
class finallydemo
{
    public static void main(String args[])
    { method1();
      System.out.println("Result :"+method2(24,0)); }
    static void method1()
    { try {
        System.out.println("In method 1");
        throw new NullPointerException(); }
      catch(Exception e)
      { System.out.println("Exception Handled: '"+e); }
      finally {
        System.out.println("In method1 finally"); }}
```


User Defined / Custom Exceptions

- Java allows the user to create their own exception.
- The mandatory requirement is that the class should be a subclass of the Exception class.
- The custom exception class should provide a constructor which takes the exception description as its argument.

```
class A extends Exception
{
    A()
    {
        super();
    }
    A(String s)
    {
        super(s);
    }
}
```

User Defined / Custom Exceptions – Example 1

```
class InvalidAgeException extends Exception
{
    InvalidAgeException(String s)
    {    super(s);    }
}

class TestCustomException1
{ static void validate(int age)throws InvalidAgeException
    { if(age<18)
        throw new InvalidAgeException("not valid");
      else
        System.out.println("welcome to vote");    }
  public static void main(String args[])
    {    try
        {    validate(13);    }
      catch(Exception m)
        {System.out.println("Exception occurred: "+m);}
      System.out.println("rest of the code...");
    }
}
```

User Defined / Custom Exceptions – Example 2

```
public class InvalidRadiusException extends Exception
{
    private double radius;
    InvalidRadiusException()
    { super("invalid radius!"); }
    InvalidRadiusException(double radius)
    { super("Invalid radius ");
      this.radius = radius;
    }
    public double getRadius() { return radius; }
}
```

User Defined / Custom Exceptions – Example 2

```
public class Circle
{ private double radius;
  private static int numberOfObjects = 0;
  Circle()
    { this(1.0); }
  Circle(double n) throws InvalidRadiusException
    { setRad(n);
      numberOfObjects++;
    }
  public void setRad(double n) throws InvalidRadiusException
    {
      if (newRadius >= 0)
        radius = n;
      else throw new InvalidRadiusException(n);
    }
  public static int getNumberOfObjects() {
    return numberOfObjects;
  }
}
```

User Defined / Custom Exceptions – Example 2

```
public class Main
{
    public static void main(String[] args)
    {
        try
        {
            Circle c1 = new Circle(5);
            c1.setRad(-5);
            Circle c2 = new Circle(0);
        }
        catch (InvalidRadiusException ex) {
            System.out.println("Invalid Radius: " + ex.getRadius());
        }

        System.out.println("Number of objects created: " +

            Circle.getNumberOfObjects());
    }
}
```

Rethrowing Exception

```
public class RethrowingExceptions
{ static void divide()
  { int x,y,z;
    try
    { x = 6 ;
      y = 0 ;
      z = x/y ;
      System.out.println(x + "/" + y + " = " + z);  }
    catch(ArithmeticException e)
    {System.out.println("Exception Caught in Divide()");
      System.out.println("Cannot Divide by Zero in Integer Division");
      throw e; } }
public static void main(String[] args)
{ System.out.println("Start of main()");
  try
  { divide(); }
  catch(ArithmeticException e)
  { System.out.println("Rethrown Exception Caught in Main()");
    System.out.println(e); } }
```

Stack Unwinding

```
import java.util.Scanner;
public class TakeInput
{
    public static void main(String[] args)
    {
        Scanner s=new Scanner(System.in);
        System.out.print("Enter an integer: ");
        int num=s.nextInt();
        System.out.println("You entered "+num);
    } }
```

Enter an integer: Java

Exception in thread "main" java.util.InputMismatchException
at java.util.Scanner.throwFor(Scanner.java:840)
at java.util.Scanner.next(Scanner.java:1461)
at java.util.Scanner.nextInt(Scanner.java:2091)
at java.util.Scanner.nextInt(Scanner.java:2050)
at TakeInput.main(TakeInput.java:7)

Stack Unwinding

- When an exception is thrown but not caught in a particular scope, the method-call stack is "unwound," and an attempt is made to catch the exception in the next outer try block. This process is called **stack unwinding**.
- Unwinding the method-call stack means that the method in which the exception was not caught terminates, all local variables in that method go out of scope and control returns to the statement that originally invoked that method.
- If a `try` block encloses that statement, an attempt is made to catch the exception. If a try block does not enclose that statement, stack unwinding occurs again.

Stack Unwinding Example

```
public class UsingExceptions
{ public static void main( String args[] )
  { try
    {   throwException(); }
    catch ( Exception exception)
    { System...( "Exception handled in main" ); } }

public static void throwException() throws Exception
{ try
  {System.out.println( "Method throwException" );
   throw new Exception();}
  catch ( RuntimeException runtimeException )
  {System...("Exception handled in method throwException");}
  finally
  {System...( "Finally is always executed" ); }
} }
```

Chained Exceptions

- Chained Exceptions allows to relate one exception with another exception, i.e one exception describes cause of another exception.
- An application often responds to an exception by throwing another exception. In effect, the first exception *causes* the second exception.

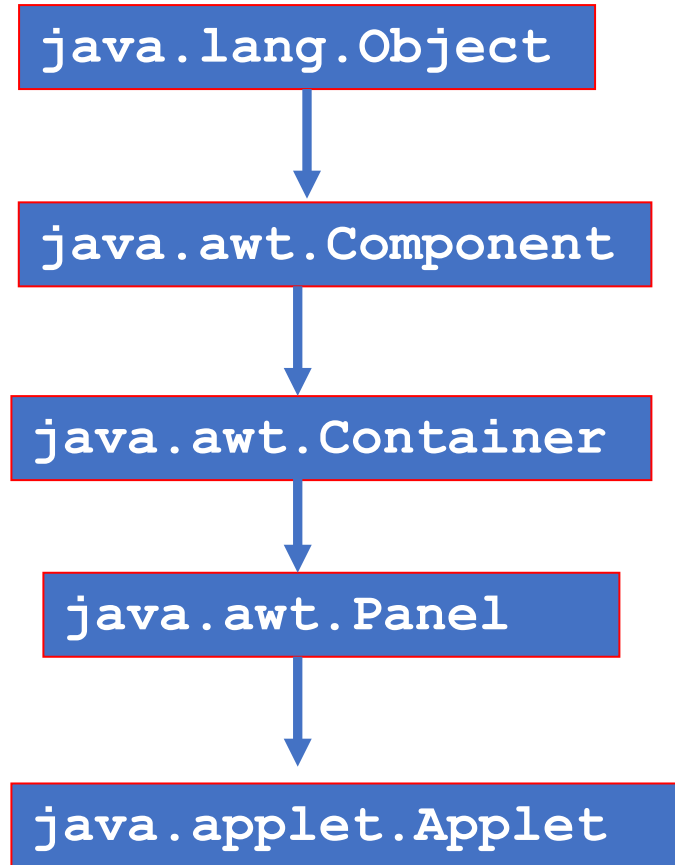
Chained Exception Example

```
public class Main
{ public static void main (String args[])throws Exception
  { int n = 20, result = 0;
    try
    { result = n/0;
      System.out.println("The result is"+result);
    }
    catch(ArithmeticException ex)
    {System...("Arithmetic exception occoured: "+ex);
      try { throw new NumberFormatException(); }
      catch(NumberFormatException ex1)
      {System...("Chained exception thrown manually: "+ex1);
      }
    }
  }
}
```

GUI Introduction

- Applet is a small program
 - can be placed on a web page
 - will be executed by the web browser
 - give web pages “dynamic content”
- Applets are Java programs that can be embedded in HTML documents
 - To run an applet you must create a .html file which references the applet
 - *Ready to Program* also will run an applet
- When browser loads Web page containing applet
 - Applet downloads into Web browser
 - begins execution
- Can be tested using appletviewer program

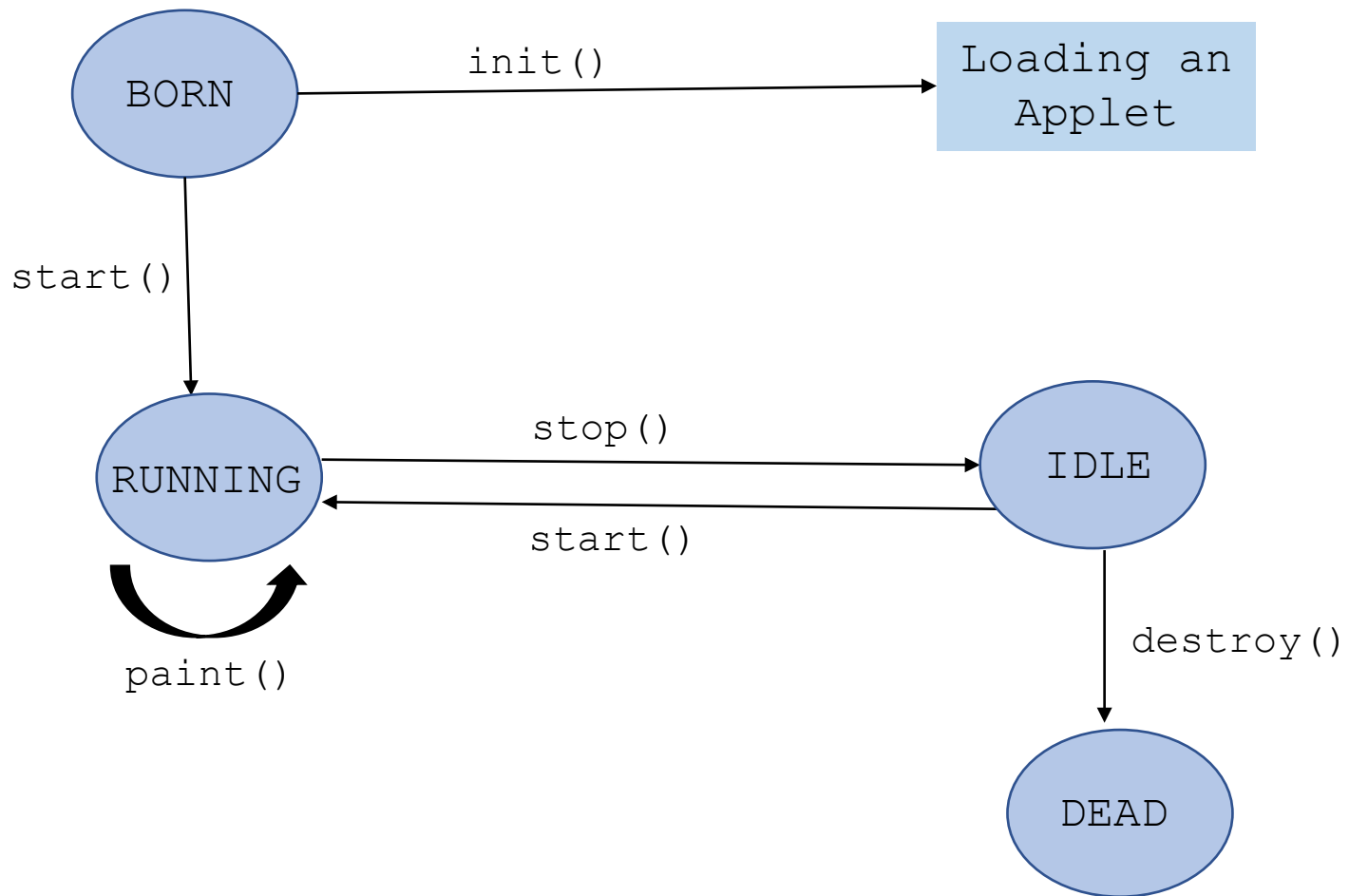
Applet class hierarchy



Applet vs Applications

Applet	Application
Does not have a main() function.	Execution begins from main()
Are not standalone programs. Needs to be embedded into webpages	Are standalone programs
Can only be executed in an appletviewer or web browser	Can be executed from command line

Applet Life Cycle



Applet Life Cycle

- `init ( )`
 - Called when applet is loaded onto user's machine. Prep work or one-time-only work done at this time.
- `start ( )`
 - Called when applet becomes visible (page called up). Called every time applet becomes visible.
- `stop ( )`
 - Called when applet becomes hidden (page loses focus).
- `paint (Graphics g)`
 - Displays output
- `destroy ( )`
 - Guaranteed to be called when browser shuts down.

Useful Methods

- `setBackground(Color c)`
- `setForeground(Color c)`
- `showStats(String s)`
- `update()` : clears the surface of the calling component to its background color and then calls `paint()`
- `repaint()` : calls `paint()`

Graphics class

- `drawArc(int x, int y, int width, int height, int startAngle, int arcAngle)`: **Draws the outline of an arc**
- `drawLine(int x1, int y1, int x2, int y2)`: **Draws a line**
- `drawPolygon(int [] x, int y[], int npoints)`: **Draws a polygon specified by the arrays of x and y coordinates.**
- `drawOval(int x, int y, int width, int height)`: **Draws an oval**
- `drawRoundRect(int x, int y, int width, int height, int arcwidth, int archeight)`: **Draws a rounded rectangle**
- `drawstring(String str, int x, int y)`: **Draws the string on the specified coordinates**
- `getColor()`: **Returns the graphic context's current color**
- `setColor(Color c)`: **Sets the drawing color**
- `getFont()`: **Returns the graphic context's current font**
- `setFont(Font c)`: **Sets the font**

Applet Program: Example 1

```
import java.applet.*;
import java.awt.*;
public class FirstApplet extends Applet
{ public void init()
    { setBackground(Color.white);
      setForeground(Color.black);
    }
  public void paint(Graphics g)
    { g.drawString("My Applet", 50, 50);
    }
}
```

How to run an applet

1. Save the file as `FirstApplet.java` and compile by using `javac`.
2. Type the following code into a text editor and save it as `FirstApplet.html`

```
<HTML><BODY>
```

```
<APPLET code = "FirstApplet.class" WIDTH = 200  
HEIGHT = 200>
```

```
</APPLET>
```

```
</BODY></HTML>
```

3. Execute the HTML file as:

```
appletviewer FirstApplet.html
```

1. Type the following code at the top of the file `FirstApplet.java`

```
/* <APPLET code = "FirstApplet.class" WIDTH = 200  
HEIGHT = 200></APPLET> */
```

2. Execute the applet as:

```
appletviewer FirstApplet.java
```

The Applet tag

```
<APPLET [CODEBASE=codebasedURL]   code = appletFile [ALT  
= alternate text] [NAME = name] WIDTH = w HEIGHT = h  
[ALIGN = alignment] [VSPACE = v] [HSPACE = hs]>  
[<PARAM NAME = attributeName VALUE = attributeValue]  
.....  
</APPLET>
```

CODEBASE: It specifies the URL of the directory where the executable class file will be searched for

ALT: It specifies the text to display in case the applet cannot be displayed.

NAME: gives a name to the applet

ALIGN: sets the alignment (LEFT, RIGHT, TOP, BOTTOM..)

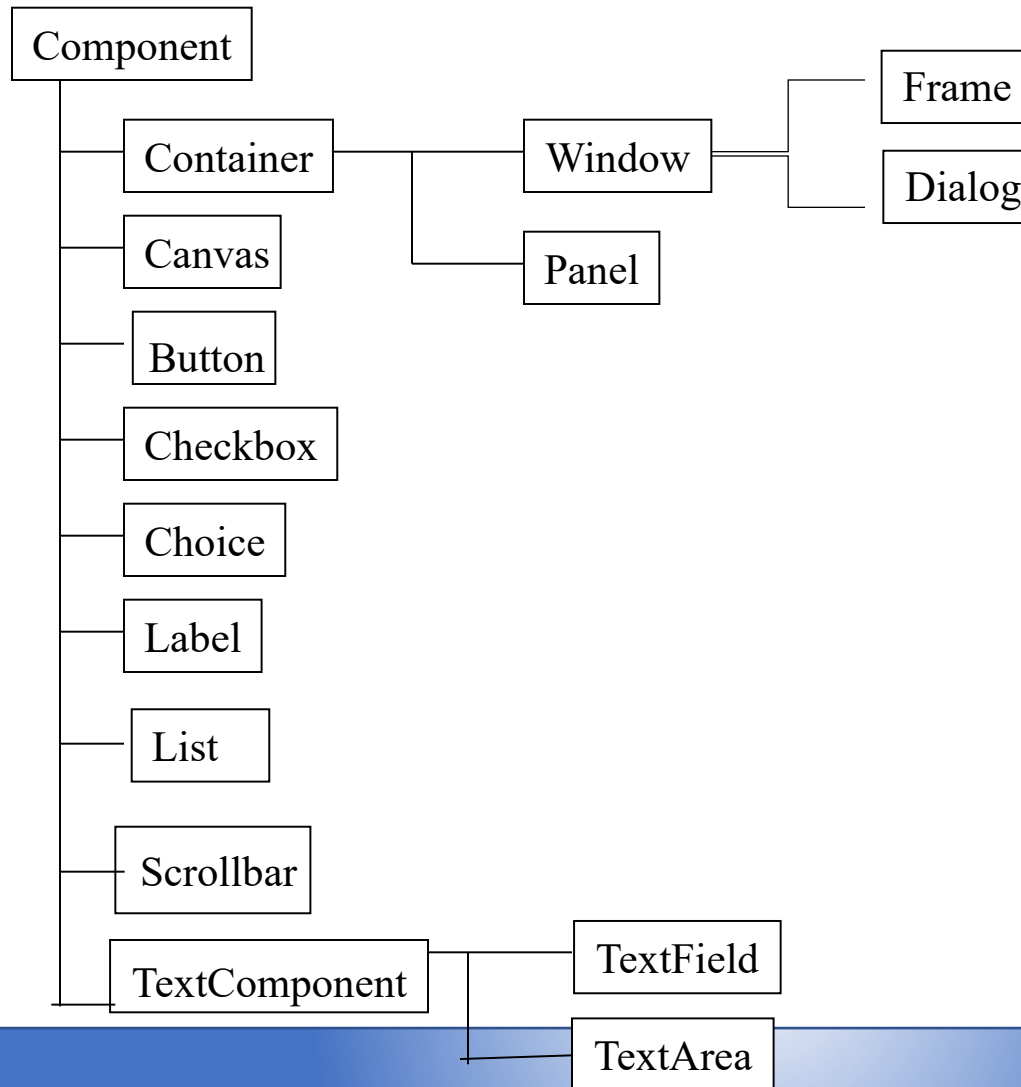
VSPACE: specifies the space (in pixels) above and below the applet.

HSPACE: specifies the space (in pixels) on each side of the applet.

PARAM: It is used to supply user defined parameters to the applet

AWT

- `java.awt` contains all classes for creating a GUI.



Component class

- Component is the superclass of most of the displayable classes defined within the AWT. It is abstract.
- The Component class defines data and methods which are relevant to all Components

```
void setBounds(int x, int y, int height, int width)
void setSize(int height, int width)
void setFont(Font f)
void setEnabled(boolean)
void setVisible(boolean)
void setForeground(Color c)
void setBackground(Color c)
```

Container class

- Container is a subclass of Component.
- Containers contain components. For a component to be placed on the screen, it must be placed within a Container
- The Container class defined all the data and methods necessary for managing groups of Components

```
void add(Component c)
Component GetComponent(int n)
Dimension getMaximumSize()
Dimension getMinimumSize()
Dimension getPreferredSize()
void remove(Component c)
void removeAll()
```


Window and Panel class

Window class

- The Window class defines a top-level Window with no Borders or Menu bar, usually used for application splash screens
- Frame defines a top-level Window with Borders and a Menu Bar. Frames are more commonly used than Windows
- Once defined, a Frame is a Container which can contain Components

Panel class

- When writing a GUI application, the GUI portion can become quite complex.
- To manage the complexity, GUIs are broken down into groups of components. Each group generally provides a unit of functionality.
- A Panel is a rectangular Container whose sole purpose is to hold and manage components within a GUI.

Other Component sub classes

Button

- `void addActionListener(ActionListener al)`
- `String getActionCommand()`
- `ActionListener[] getActionListeners()`
- `String paramString()`
- `void removeActionListener(ActionListener al)`
- `void setActionCommand(String command)`

Label

- `String getText()`
- `void setText(String s)`

Other Component sub classes

List

- `void add(String item)`
- `Void add(String item, int index)`
- `void addActionListener(ActionListener al)`
- `void addItemListener(ItemListener il)`
- `void deselect(int index)`
- `ActionListener[] getActionListeners()`
- `ItemListener[] getItemListeners()`
- `String [] getItems()`
- `int getSelectedIndex()`
- `int[] getSelectedIndexes()`
- `String getSelectedItem()`
- `String[] getSelectedItems()`
- `void remove(int position)`
- `void remove(String item)`
- `void removeActionListener(ActionListener al)`
- `void removeItemListener(ItemListener il)`
- `void removeAll()`

Other Component sub classes

TextField

- `void addActionListener(ActionListener al)`
- `ActionListener[] getActionListeners()`
- `int getColumns()`
- `String paramString()`
- `void setColumns(int columns)`
- `void setText(String s)`
- `void removeActionListener(ActionListener al)`

Checkbox

- `void addItemListener(ItemListener il)`
- `CheckboxGroup getCheckboxGroup()`
- `String getLabel()`
- `ItemListener[] getItemListeners()`
- `boolean getState()`
- `String paramString()`
- `void removeItemListener(ItemListener il)`
- `void setCheckboxGroup(CheckboxGroup ckgb)`
- `void setLabel(String label)`
- `void setState(boolean state)`

Example 1

```
/*<applet code = Demo.class width=400 height=200></applet>*/
import java.applet.*;
import java.awt.*;
public class Demo extends Applet
{ public void init()
    { setBackground(Color.white);
      setForeground(Color.black);
      Button red = new Button("RED");
      label l1 = new Label("COLOUR");
      Checkbox chk = new Checkbox("Yes", null, false);
      add(red);
      add(l1);
      add(chk);
    }
  public void paint(Graphics g)
    { g.drawString("My Applet", 50, 50);
    }
}
```

Text on the button

Checkbox text

Checkbox group

Checkbox on

Event classes and Listeners

Events Classes	Description	Event Listeners
ActionEvent	Generated when a component defined action occurred	ActionListener
AdjustmentEvent	When scrollbar is adjusted	AdjustmentListener
ContainerEvent	When an element is added or removed from a container	ContainerListener
FocusEvent	When a component gains or loses focus	FocusListener
KeyEvent	When input is received from keyboard	KeyListener
ComponentEvent	When a component is moved or changed size	ComponentListener
MouseEvent	An event indicating mouse action	MouseListener/ MouseMotionListener
ItemEvent	Generated when an item state changes	ItemListener
TextEvent	When an object's text changes	TextListener
WindowEvent	When a window has changed its status	WindowListener

Different component sub classes and their Events

Components	Event	Events Type	Event Listeners	Method Name
Button	Click	ActionEvent	ActionListener	actionPerformed()
	Focus gained/ focus lost	FocusEvent	FocusListener	focusGained() / focusLost()
Checkbox	Selection / Deselection	ItemEvent	ItemListener	itemStateChanged()
Choice	Selection / Deselection	ItemEvent	ItemListener	itemStateChanged()
List	Selection / Deselection	ItemEvent	ItemListener	itemStateChanged()
	Double Clicking an item	ActionEvent	ActionListener	actionPerformed()
TextField	Focus gained/ focus lost	FocusEvent	FocusListener	focusGained() / focusLost()
	Press enter key	ActionEvent	ActionListener	actionPerofrmed()
	Text Changes	TextEvent	TextListener	textValueChanged()

Different component sub classes and their Events

Components	Event	Events Type	Event Listeners	Method Name
TextArea	Focus gained / focus lost	FocusEvent	FocusListener	focusGained() / focusLost()
	Text Changes	TextEvent	TextListener	textValueChanged()
Scrollbar	Change the value of scrollbar	AdjustmentEvent	AdjustmentListener	adjustmentValueChanged()
Frame	Mouse motion	MouseEvent	MouseMotionListener	mouseMoved() mouseDragged()
	Window event	WindowEvent	WindowListener	windowActivated() windowClosed() windowClosing() windowDeactivated() windowOpened()

Event Methods

Event	Events Type	Event Listeners	Method Name
Key events	KeyEvent	KeyListener	keyPressed() keyReleased() keyTyped()
Mouse events	MouseEvent	MouseListener	mouseClicked() mouseEntered() mouseExited() mousePressed() mouseReleased()

Example 2

```
/*<applet code = Demo.class width=400 height=200></applet>*/
import java.applet.*;
import java.awt.*;
public class Demo extends Applet implements ActionListener
{ Button red, white, blue;
  Label hit;
  public void init()
  { red = new Button("RED");
    white = new Button("WHITE");
    blue = new Button("BLUE");
    hit = new Label("Hit button to change screen colour");
    add(red); add(white); add(blue); add(hit);
    red.addActionListener(this); blue.addActionListener(this);
    white.addActionListener(this); }
  public void actionPerformed(ActionEvent ae)
  { String str = ae.getActionCommand();
    if(str.equals("RED")) { setBackColor(Color.red); }
    else if(str.equals("BLUE"))
      { setBackColor(Color.blue); }
    else if(str.equals("WHITE"))
      { setBackColor(Color.white); } repaint(); } }
```

Example 3

student registration

Student details

Name:

age:

Sex(MF): ☒ Male ☐ Female

Address:

Course:

Semester:

Student details saved!

Example 4

```
/*<applet code = Demo.class width=400 height=200></applet>*/
import java.applet.*;
import java.awt.*;
public class Demo extends Applet implements ActionListener,
TextListener, FocusListener
{
    Label l; Button b; TextField tf;

    public void init()
    {
        tf = new TextField("TextField");
        l = new Label("Make Selection:");
        b = new Button("Submit");
        add(tf);    add(l);    add(b);
        b.addFocusListener(this);
        tf.addFocusListener(this);
        tf.addActionListener(this);
        tf.addTextListener(this);
    }
}
```

Example 4

```
public void focusGained(FocusEvent e)
{ if(e.getSource()==tf)
    l.setText("Focus Gained by TextBox");
  else if(e.getSource()==b)
    l.setText("Focus Gained by Button");
}

public void focusLost(FocusEvent e)
{ l.setText("Focus Lost"); }

public void actionPerformed(ActionEvent e)
{ l.setText("Action Event"); }

public void textValueChanged(TextEvent e)
{ l.setText("Trying to change text"); }
}
```

Adapter Classes

- For listener interfaces containing more than one event handling methods JDK defines a corresponding adapter class.
- Java adapter classes provide the default implementation of listener interfaces.
- If you inherit the adapter class, you will not be forced to provide the implementation of all the methods of listener interfaces.
- So it saves code.

Listener Interface	Adapter Classes
WindowListener	WindowAdapter
KeyListener	KeyAdapter
MouseListener	MouseAdapter
MouseMotionListener	MouseMotionAdapter
FocusListener	FocusAdapter
ComponentListener	ComponentAdapter
ContainerListener	ContainerAdapter

Adapter Classes: Example 10 (Applet)

```
/*.....*/  
import ..  
public class Example extends Applet  
{ int xcord, ycord;  
  public void init()  
  {   addMouseMotionListener(new MouseDemo(this)); }  
  public void paint(Graphics g)  
  { g.drawString("(" + xcord + ", " + ycord + ")", xcord, ycord); }  
} }  
public MouseDemo extends MouseAdapter  
{ Example e;  
  MouseDemo(Example e) { this.e=e; }  
  public void mouseClicked(MouseEvent me)  
  {   d.xcord = me.getX();   d.ycord = me.getY();  
      d.repaint();  
  }  
}
```